Internship Project Report

AQI Forecasting System for Islamabad

An End-to-End Machine Learning System using Feast Feature Store, FastAPI, Streamlit, and GitHub Actions

Submitted By: Maryam Naseem

Internship Track: Data Science

Track Lead: Muhammad Mobeen

Date: June 7, 2026

1. Executive Summary

I developed an end-to-end Air Quality Index forecasting system for Islamabad. The system predicts AQI for the next 24, 48, and 72 hours using real AQI and weather data. Instead of building only a standalone notebook model, I focused on creating a complete machine learning system that includes ingestion, cleaning, feature engineering, feature store integration, model training, model registry, explainability, backend serving, dashboard visualization, and automation through GitHub Actions.

The final project uses AQICN and Open-Meteo data sources, stores processed features using Feast, trains Ridge Regression, Random Forest, and XGBoost models, selects the best model for each forecast horizon, serves predictions through FastAPI, and displays results in a Streamlit dashboard. I also generated SHAP plots to explain model predictions and added health advisory logic based on AQI categories.

- 24-hour forecast best model: Random Forest with RMSE 6.3263, MAE 4.8249, and R2 0.6822.
- 48-hour forecast best model: XGBoost with RMSE 6.8683, MAE 5.4897, and R2 0.6203.
- 72-hour forecast best model: XGBoost with RMSE 9.9377, MAE 8.3384, and R2 0.2280.
- The complete project is version-controlled on GitHub and includes an automated hourly ingestion workflow.

2. Introduction and Problem Statement

Air pollution is an important environmental and public health problem. AQI values can change due to pollutants, weather conditions, traffic, industrial activity, and seasonal patterns. Predicting AQI in advance can help people make better health and outdoor activity decisions, especially sensitive groups such as children, elderly people, and people with breathing problems.

The problem I addressed in this project was to build a practical AQI forecasting system for Islamabad that can collect real data, convert it into machine learning features, train forecasting models, and display useful predictions and health advice through a dashboard. My goal was to make the system close to a real-world ML workflow, not just a training script.

3. Project Objectives

- Fetch real AQI and weather data from external APIs.
- Clean and prepare data for machine learning.
- Generate time-based, lag-based, rolling, pollutant, weather, and ratio features.
- Use a feature store to organize and retrieve features consistently.
- Train and compare multiple machine learning models for 24h, 48h, and 72h AQI forecasting.
- Save models and metadata in a model registry.

- Build a FastAPI backend to serve predictions and model information.
- Build a Streamlit dashboard for interactive visualization.
- Generate SHAP explanations for model interpretability.
- Automate ingestion and pipeline execution using GitHub Actions.
- Document problems faced during development and how I resolved them.

4. Requirement Mapping

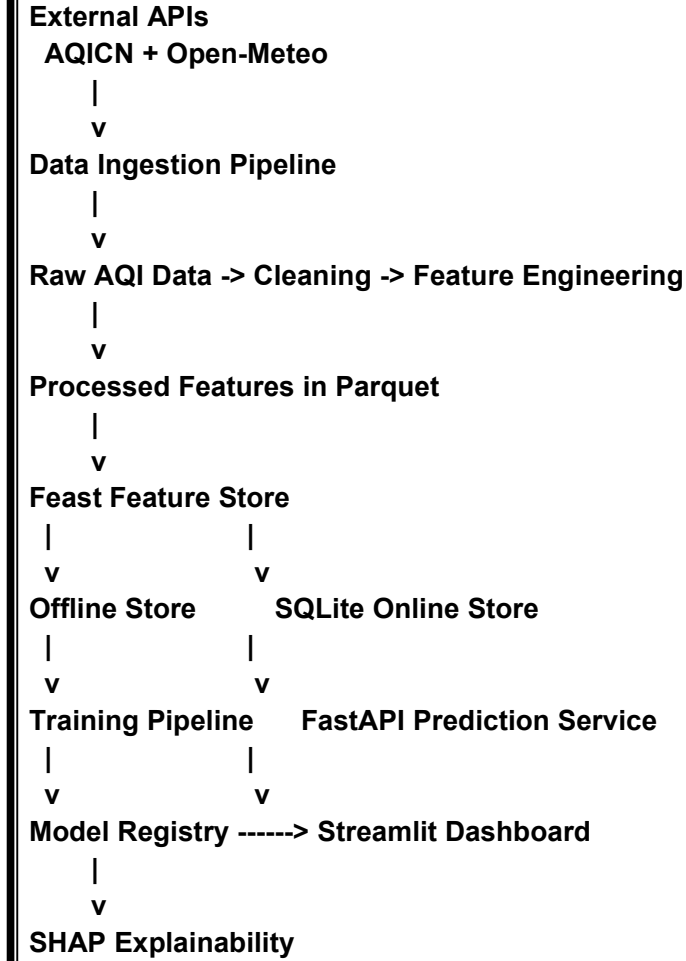
Requirement	Implementation	Status
Real-time AQI and weather data	AQICN and Open-Meteo APIs	Completed
Historical data for training	Raw and processed Parquet datasets with thousands of records	Completed
Feature engineering	Time, lag, rolling, ratio, pollutant, and weather features	Completed
Feature store	Feast offline store and SQLite online store	Completed
Model training	Ridge, Random Forest, and XGBoost models	Completed
Model registry	Models and metadata saved in artifacts/models	Completed
3-day forecast	24h, 48h, and 72h predictions	Completed
Backend API	FastAPI endpoints for predictions, metrics, health advice, drift, and retraining	Completed
Dashboard	Streamlit dashboard with charts, metrics, forecast cards, SHAP, and advisory	Completed
Explainability	SHAP summary and bar plots for all horizons	Completed
Automation	GitHub Actions hourly ingestion workflow	Completed

5. Tools and Technologies

Category	Tools / Libraries
Programming Language	Python
Data Processing	Pandas, NumPy
Machine Learning	Scikit-learn, XGBoost
Feature Store	Feast with local provider, file offline store, and SQLite online store
Model Registry	Local artifact registry in artifacts/models
Explainability	SHAP and Matplotlib
Backend	FastAPI and Uvicorn
Dashboard	Streamlit and Plotly
Automation	GitHub Actions
Data Storage	Parquet files
Version Control	Git and GitHub

6. System Architecture

The system was designed as a pipeline in which each component has a clear responsibility. The external APIs provide the data. The ingestion module collects and stores raw data. Cleaning and feature engineering modules prepare the data for machine learning. Feast manages feature registration, offline training retrieval, and online serving retrieval. The training pipeline compares models and saves the best models in the registry. The FastAPI backend loads the selected model and features, while Streamlit provides the user interface.



7. Data Sources and Dataset Preparation

I used real data sources for this project. AQI data was collected from AQICN, and weather data was collected from Open-Meteo. The project stores raw data and processed data in Parquet format because Parquet is efficient for tabular data and works well with pipeline-based processing.

- Raw data path: data/raw/raw_aqi_data.parquet
- Processed data path: data/processed/processed_aqi_data.parquet
- Main city/entity: Islamabad
- Entity key used in Feast: city = islamabad

- The dataset included thousands of processed records after historical data restoration and pipeline execution.

One important decision I made was to avoid relying on fake data for the final system. I used real API-based data and corrected the pipeline so that the system could continue growing data over time through automation.

8. Data Ingestion, Cleaning, and Feature Engineering

8.1 Data Ingestion

I started by building the ingestion layer. The ingestion pipeline fetches AQI data and weather data, normalizes the response, and appends the latest record to the raw Parquet dataset. I also handled stale timestamps by overriding old API timestamps with the current system time when required.

8.2 Cleaning and Preprocessing

After ingestion, I applied cleaning logic to handle missing values, timestamp parsing, data type conversion, duplicate rows, and outliers. Cleaning was important because raw API responses can contain missing pollutant values or inconsistent formats.

8.3 Feature Engineering

Feature engineering was one of the most important parts of the project because AQI forecasting is a time-dependent problem. I created features that help the model understand temporal patterns, recent AQI behavior, rolling trends, and pollutant relationships.

Feature Group	Examples
Time features	hour, day_of_week, day_of_month, month, is_weekend, season
Cyclical features	hour_sin, hour_cos, month_sin, month_cos
Rolling features	AQI, PM2.5, and PM10 rolling means and standard deviations
Lag features	aqi_lag_1h, aqi_lag_3h, aqi_lag_6h, aqi_lag_12h, aqi_lag_24h
Change features	aqi_change and aqi_pct_change
Ratio features	pm25_pm10_ratio and o3_no2_ratio

9. Feast Feature Store Integration

I used Feast as the feature store layer. The feature store helped me organize the project like a real ML system where training and serving both use consistent features. The offline store was used for historical feature retrieval during training, while the SQLite online store was used for retrieving the latest features during prediction serving.

9.1 Why I Used Feast Instead of Hopsworks

The original project direction included feature store concepts, and Hopsworks was considered earlier. However, I decided to use Feast for the final implementation because it was more suitable for local development, GitHub-based reproducibility, and internship evaluation. Hopsworks is a managed platform and often requires cloud setup, external credentials, and account-level configuration. Since I wanted the evaluator to run and inspect the system locally, Feast was a better fit.

- Feast is open-source and can run locally without depending on a managed cloud account.
- It supports a file-based offline store, which matches my Parquet-based pipeline.
- It supports a SQLite online store, which works well for local prediction serving.
- It allowed me to demonstrate real feature store concepts such as feature views, entities, offline retrieval, online retrieval, and materialization.
- It made the project more reproducible for GitHub, local testing, and evaluation.

9.2 Feast Components Used

Component	Implementation
Project	aqi_islamabad
Entity	city
Feature View	aqi_islamabad_features
Offline Store	File store using processed_aqi_data.parquet
Online Store	SQLite online_store.db
Materialization	Feast materialize and materialize-incremental for online feature availability

10. Model Training, Evaluation, and Model Registry

I trained three models for each forecast horizon: Ridge Regression, Random Forest, and XGBoost. I evaluated models using RMSE, MAE, and R2. Lower RMSE and MAE are better, while a higher R2 score indicates better explained variance. The model with the strongest metric performance for each horizon was selected as the best model and saved in the registry.

Model	Horizon	RMSE	MAE	R2
Ridge	24h	11.5651	9.0745	-0.0622
Random Forest	24h	6.3263	4.8249	0.6822
XGBoost	24h	6.5002	4.9043	0.6645
Ridge	48h	8.4406	6.9728	0.4265
Random Forest	48h	8.1753	6.5973	0.4620
XGBoost	48h	6.8683	5.4897	0.6203
Ridge	72h	11.9215	9.6371	-0.1110
Random Forest	72h	10.0043	8.3957	0.2176
XGBoost	72h	9.9377	8.3384	0.2280

Forecast Horizon	Selected Best Model	RMSE	MAE	R2
24h	Random Forest	6.3263	4.8249	0.6822
48h	XGBoost	6.8683	5.4897	0.6203
72h	XGBoost	9.9377	8.3384	0.2280

The results show that shorter-horizon forecasts performed better than the 72-hour forecast. This is expected because AQI becomes harder to predict as the forecast horizon increases. I selected models based on actual evaluation metrics rather than assuming that one algorithm would always perform best.

10.1 Model Registry

The active model registry is stored in artifacts/models. It contains model files, best model selection files, and metadata JSON files. This helped me keep track of which model was selected for each horizon and what performance metrics were achieved.

- best_24h.json, best_48h.json, and best_72h.json store selected best model names.
- Joblib files store trained model objects.
- JSON metadata files store metrics, feature names, training timestamp, and model path.
- The final selected models were Random Forest for 24h, XGBoost for 48h, and XGBoost for 72h.

11. FastAPI Backend and Streamlit Dashboard

After training and saving the models, I built a FastAPI backend to serve predictions and a Streamlit dashboard to visualize the system output. The backend loads the selected model for the requested horizon, retrieves feature values, generates a prediction, and returns the result with a health advisory.

Endpoint	Method	Purpose
/api/v1/predict	POST	Predict AQI for 24h, 48h, or 72h horizon
/api/v1/health-advice	GET	Return health advisory based on latest AQI
/api/v1/metrics	GET	Return model performance metrics
/api/v1/drift-status	GET	Return latest drift monitoring status
/api/v1/retrain	POST	Trigger model retraining

The Streamlit dashboard includes pages for current AQI, weather, forecast cards, model metrics, SHAP explainability, drift status, health advisory, and retraining. I improved the dashboard design and fixed several UI issues such as the broken sidebar image, pollutant chart validation, and request timeout during retraining.

12. SHAP Explainability

To make the model predictions more interpretable, I added SHAP explainability. SHAP helped me identify which features had the most influence on model predictions. I generated both summary plots and bar plots for all three forecast horizons.

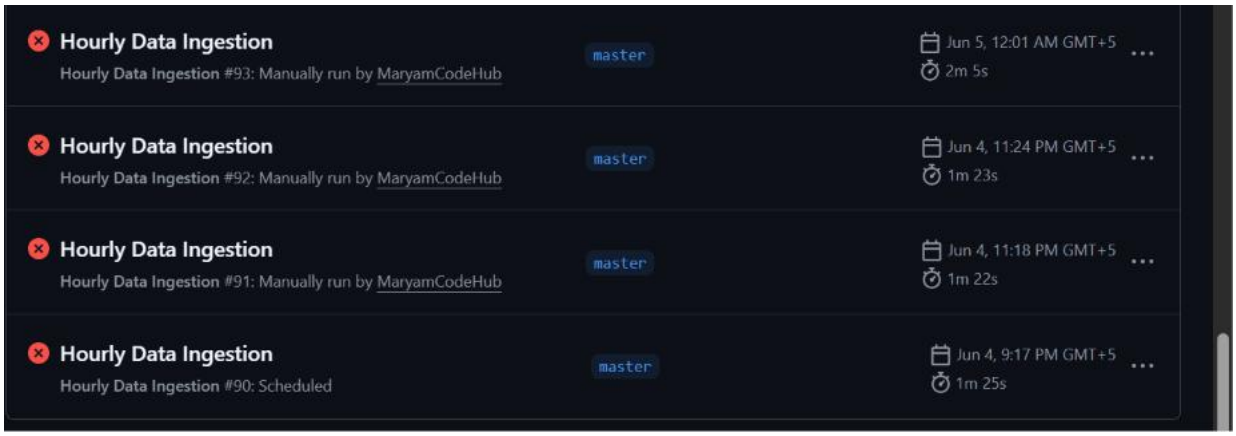
Horizon	SHAP Summary Plot	SHAP Bar Plot
24h	shap_summary_24h.png	shap_bar_24h.png
48h	shap_summary_48h.png	shap_bar_48h.png
72h	shap_summary_72h.png	shap_bar_72h.png

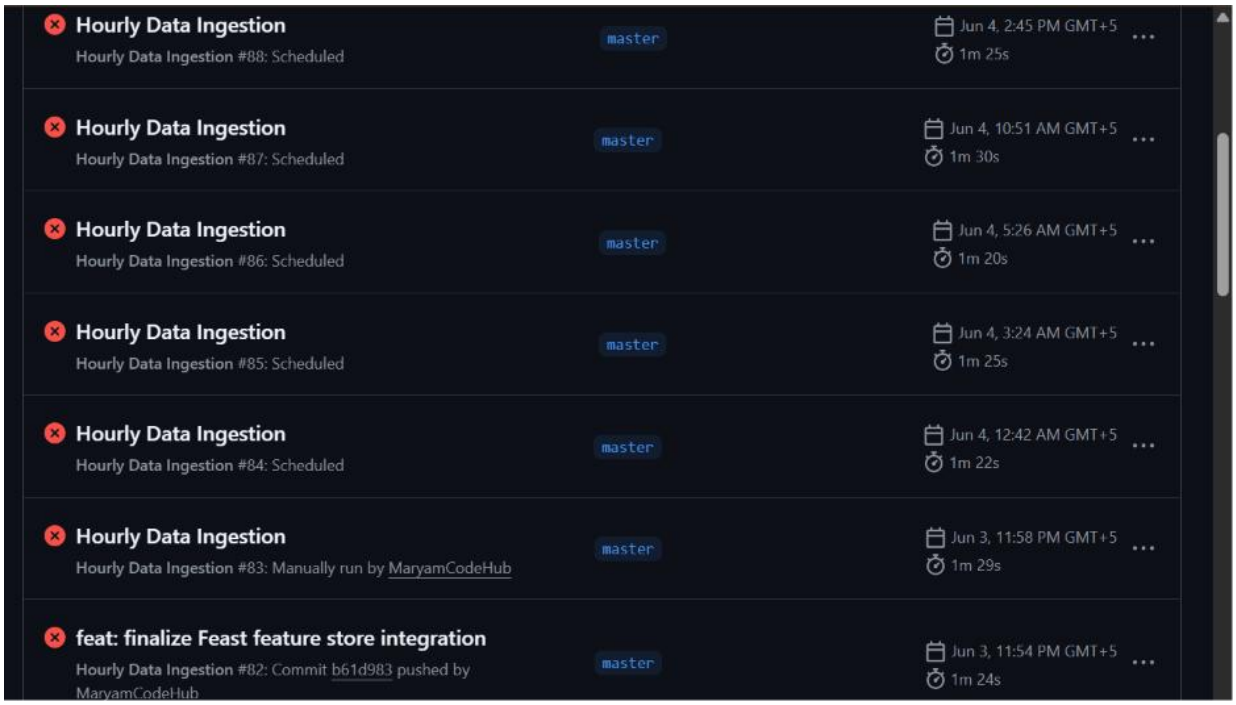
SHAP plots are also useful in the final presentation because they show that the project includes model explainability and not just prediction output.

13. GitHub Actions Automation

I used GitHub Actions to automate the data ingestion workflow. The workflow file is .github/workflows/hourly-ingest.yml. The workflow runs on schedule and can also be triggered manually. It installs dependencies, runs the ingestion pipeline, updates data files, checks whether enough data is available, and commits updated artifacts when required.

During development, several GitHub Actions runs failed. I used these failures as part of the debugging process. Instead of ignoring them, I analyzed logs, fixed the root causes, and reran the workflow until the pipeline became stable.





GitHub Actions failed runs during debugging phase

14. Problems Faced and How I Resolved Them

This was the most important learning part of the project. I faced multiple issues related to data pipeline design, feature store materialization, GitHub Actions, model paths, dashboard behavior, dependencies, and Git workflow. I documented the major problems below because they show the actual development process and the debugging steps I followed.

Problem Faced	Cause	How I Resolved It
1. GitHub Actions failed repeatedly	Workflow steps were not fully aligned with the project structure. Some expected folders and files were missing or generated in different locations.	I analyzed the workflow logs, removed the problematic step, added required folder creation such as docs/reports, and pushed fixes until scheduled workflow runs started passing.
2. Feast materialization failed with SQLite table error	The Feast online store table was missing or not created correctly for the feature view.	I reset the Feast registry and SQLite online store, reapplied feature definitions using feast apply, adjusted the serialization configuration, materialized again, and verified online feature retrieval.
3. Model artifacts were being saved in confusing locations	Older config values pointed models to models while the updated registry used artifacts/models.	I synchronized config/config.yaml and configs/config.yaml and standardized the active model registry as artifacts/models.
4. Health advisory showed wrong AQI category for decimal values	A decimal value such as 50.8 did not match the configured integer category ranges cleanly.	I updated the advisory logic so decimal AQI values are handled correctly and mapped to the expected category.
5. Retrain button timed out	The dashboard request timeout was too short compared to training time.	I increased the retrain timeout and improved the dashboard message so the user knows retraining can take time.
6. Pollutant chart looked blank or strange	The latest record had missing pollutant values, so the chart had invalid or empty data.	I updated the dashboard to validate numeric values and use the latest available valid pollutant row.
7. Git push failed with HTTP 408	Large artifacts and an unstable connection caused remote push timeout.	I increased Git HTTP settings and pushed again successfully.

14.1 What I Learned from Debugging

The debugging process taught me that real ML projects usually fail first at integration points, not only at model training. I learned to read logs carefully, isolate the failing component, test one fix at a time, and verify the system after

every change. I also learned the importance of keeping the project structure clean and keeping configuration files synchronized.

15. Limitations

- Forecast quality depends on external API reliability and data freshness.
- Some pollutant fields may be missing from real-time API responses.
- The 72-hour forecast is more difficult and has lower R2 compared to 24h and 48h horizons.
- The current feature store uses a local Feast setup with SQLite, which is suitable for local development and evaluation but not a large-scale production deployment.
- Dashboard retraining is currently synchronous and can take time.
- Commercial AQI platforms may use different sensors, proprietary models, and additional data sources, so their forecasts can differ from this project.

Deployment Limitation: Render Free Instance Cold Start

After deploying the FastAPI backend on Render and connecting it with the Streamlit dashboard, I noticed that the first request sometimes took longer to respond. This happened because the backend was deployed on a free Render instance, which can spin down after a period of inactivity. When the deployed Streamlit dashboard sends the first request after inactivity, the backend may take some time to wake up.

Initially, this caused a request timeout on the Streamlit dashboard. However, after reloading the app, the backend responded successfully and the dashboard loaded correctly. This confirmed that the deployment configuration and API URL integration were correct, and the delay was related to the free hosting cold start behavior rather than an application logic error.

In a production environment, this limitation can be solved by using a paid always-on backend service, increasing the API request timeout, or moving long-running tasks such as model retraining to a background worker or scheduled pipeline

16. Future Improvements

- Deploy FastAPI and Streamlit dashboard to a cloud platform.
- Add support for multiple cities instead of Islamabad only.
- Use future weather forecast variables for better 48h and 72h predictions.

- Add deeper model monitoring and data drift visualization.
- Use asynchronous background jobs for retraining.
- Add notification alerts for unhealthy or hazardous AQI levels.
- Add Docker support for easier deployment.
- Add unit tests for API routes, feature engineering, and model registry.
- Experiment with LSTM, GRU, or other time-series models for longer horizons.

17. Screenshot Checklist and Appendix

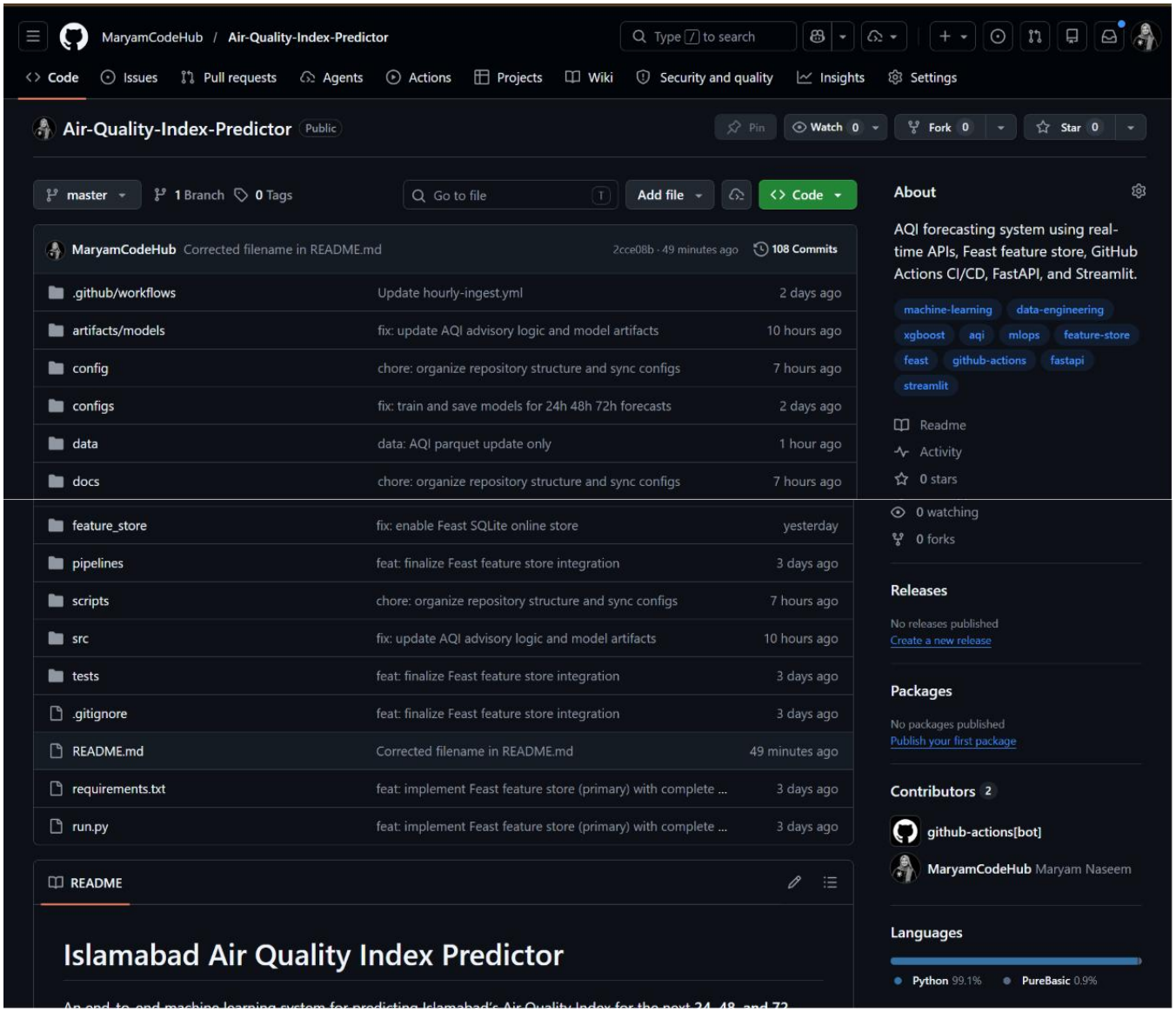
Appendix A: Important Commands Used

- python pipelines/run.py ingest
- python pipelines/run.py features
- python pipelines/run.py train
- python pipelines/run.py explain
- python pipelines/run.py drift
- python -m uvicorn src.api.main:app --host 127.0.0.1 --port 8000
- python -m streamlit run src/dashboard/app.py
- feast apply
- feast materialize 2025-09-25T00:00:00 2026-06-05T23:59:59

Appendix B: Final Selected Models

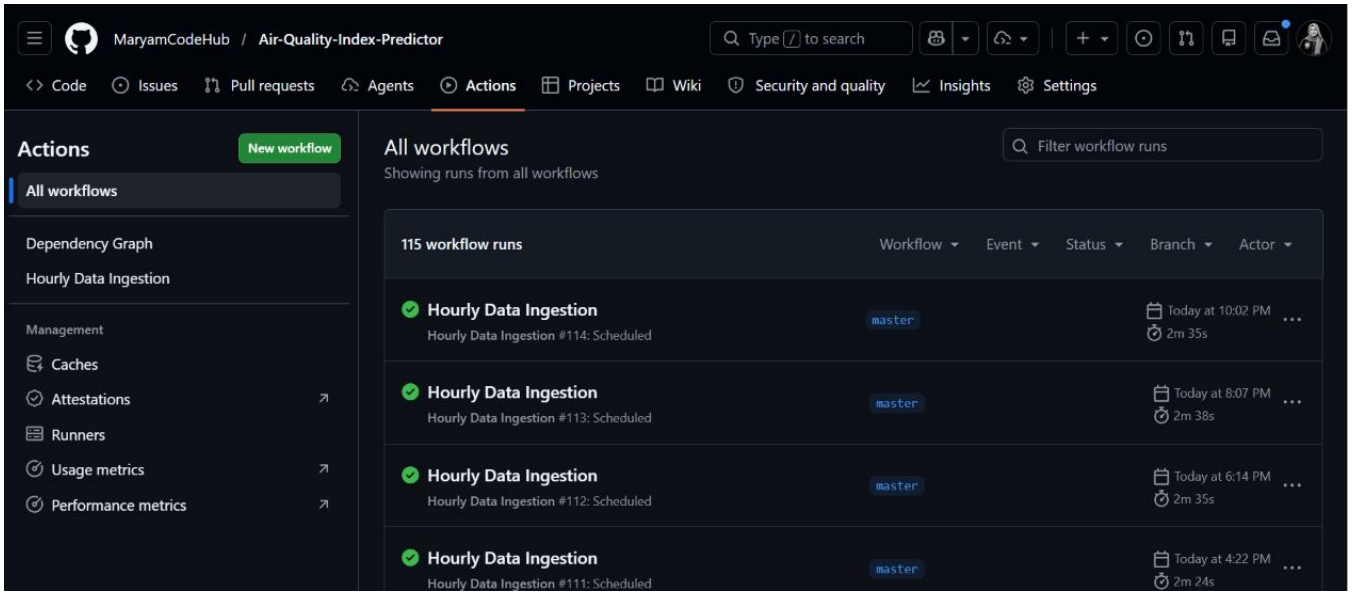
Horizon	Best Model	Model File
24h	Random Forest	artifacts/models/random_forest_24h.joblib
48h	XGBoost	artifacts/models/xgboost_48h.joblib
72h	XGBoost	artifacts/models/xgboost_72h.joblib

I. Github Repo Project Structure



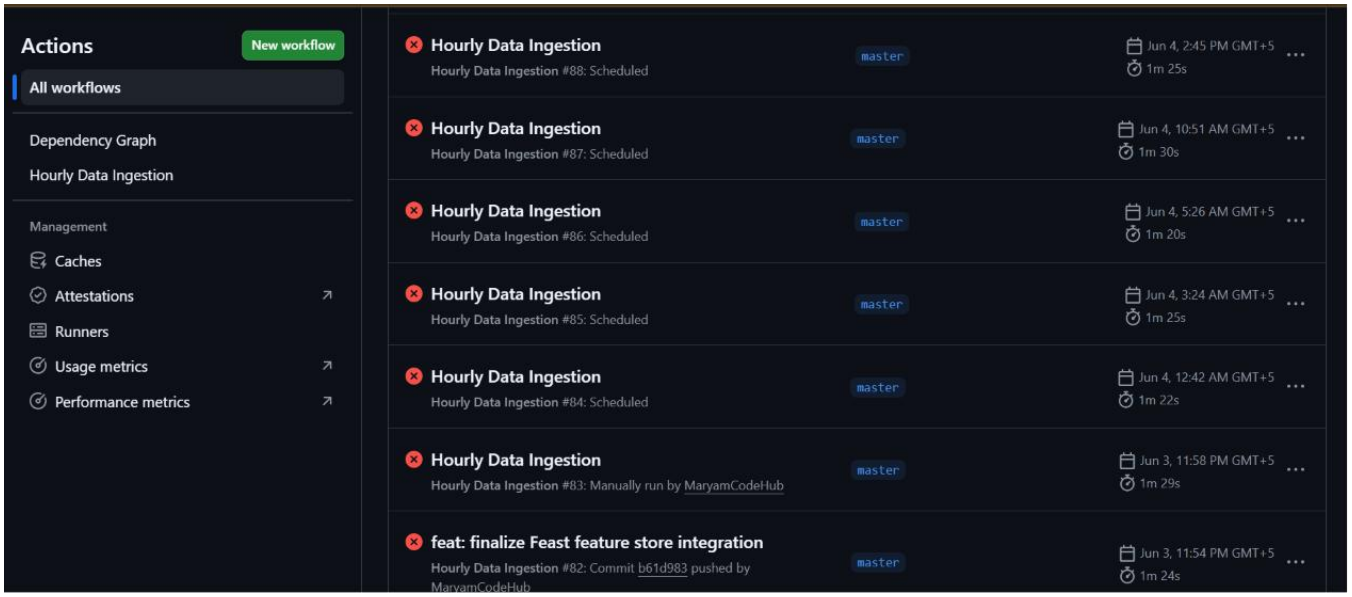
II. Github actions success

Successful GitHub Actions runs after fixing pipeline, dependency, and repository configuration issues.This Figure shows successful GitHub Actions runs after debugging and stabilizing the hourly data ingestion workflow. This confirms that the automated pipeline was able to run from GitHub and update the project data successfully.



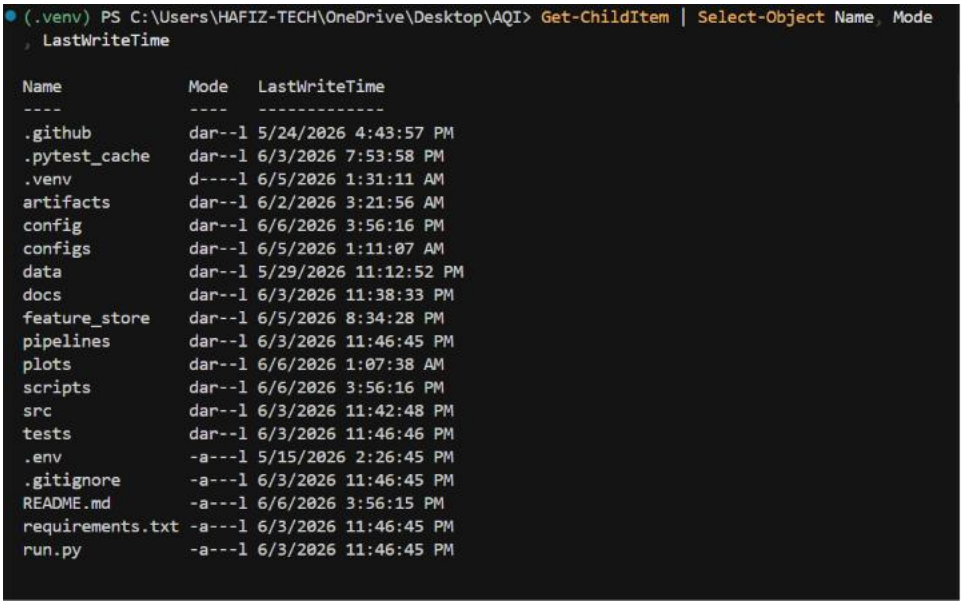
III. Github actions failed runs

Earlier failed GitHub Actions runs during debugging and pipeline stabilization.It shows earlier failed GitHub Actions workflow runs during the development phase. These failures occurred while the pipeline was being stabilized, mainly because of data path issues, configuration mismatch, dependency setup, and missing/insufficient processed data. I debugged these issues by checking workflow logs, correcting configuration paths, syncing config files, fixing data ingestion and feature engineering steps, and validating the pipeline locally before pushing updates.



IV. Final Project Structure

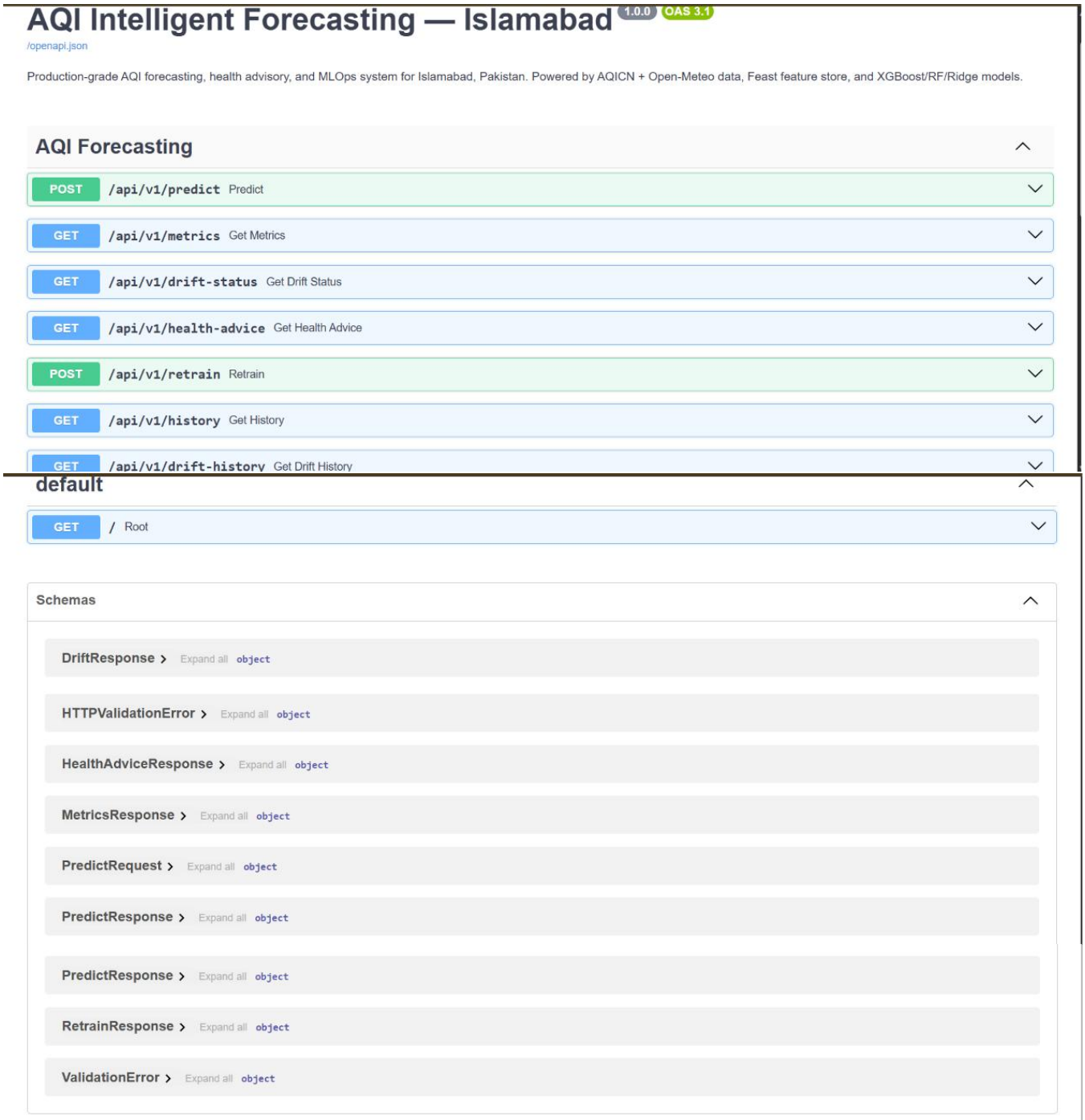
Final organized project repository structure after cleanup and except the documentation updates.



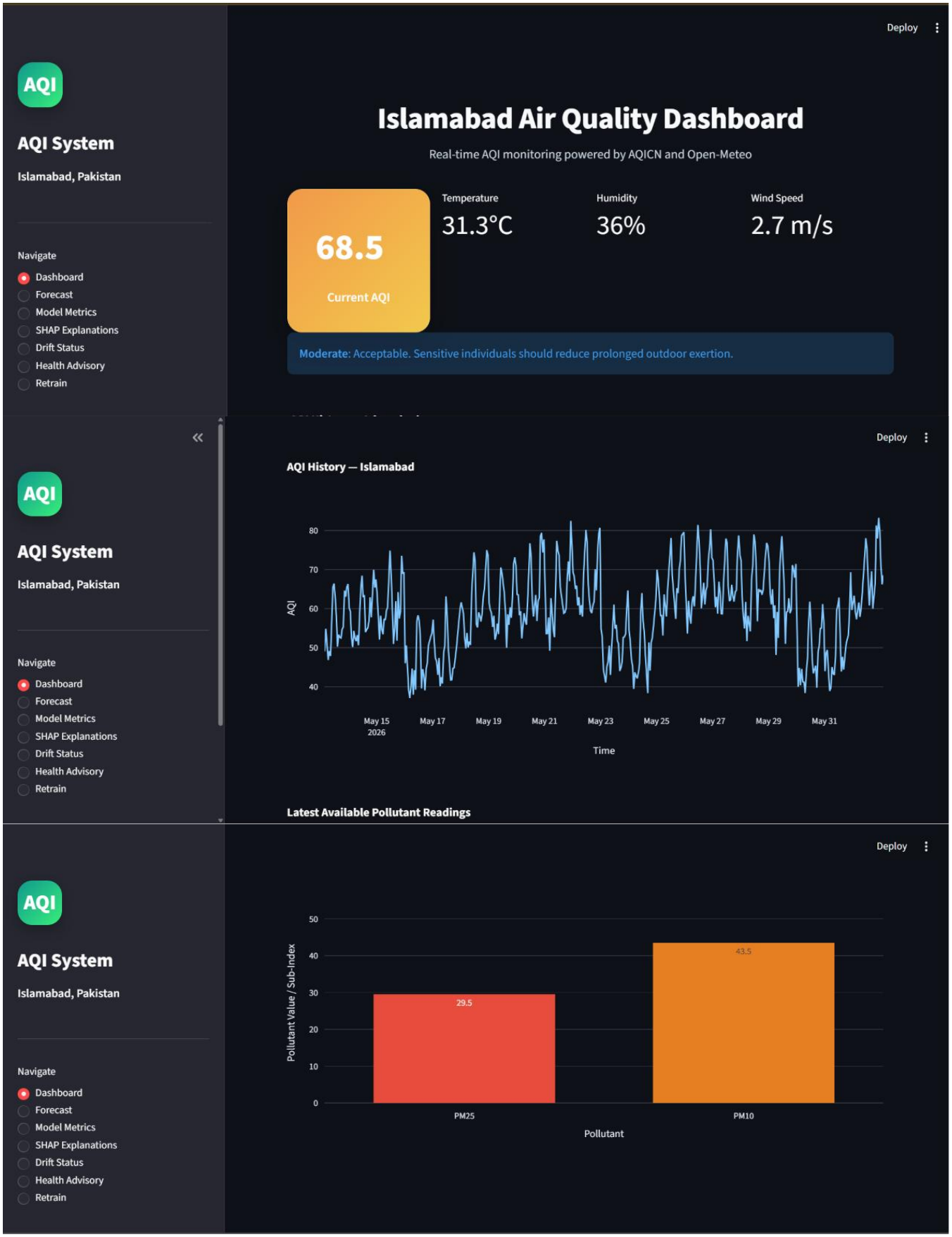
```
(.venv) PS C:\Users\HAFIZ-TECH\OneDrive\Desktop\AQI> Get-ChildItem | Select-Object Name, Mode
, LastWriteTime

Name             Mode   LastWriteTime
----             ----   -------------
.github          dar--l 5/24/2026 4:43:57 PM
.pytest_cache    dar--l 6/3/2026 7:53:58 PM
.venv            d----l 6/5/2026 1:31:11 AM
artifacts        dar--l 6/2/2026 3:21:56 AM
config           dar--l 6/6/2026 3:56:16 PM
configs          dar--l 6/5/2026 1:11:07 AM
data             dar--l 5/29/2026 11:12:52 PM
docs             dar--l 6/3/2026 11:38:33 PM
feature_store    dar--l 6/5/2026 8:34:28 PM
pipelines        dar--l 6/3/2026 11:46:45 PM
plots            dar--l 6/6/2026 1:07:38 AM
scripts          dar--l 6/6/2026 3:56:16 PM
src              dar--l 6/3/2026 11:42:48 PM
tests            dar--l 6/3/2026 11:46:46 PM
.env             -a---l 5/15/2026 2:26:45 PM
.gitignore       -a---l 6/3/2026 11:46:45 PM
README.md        -a---l 6/6/2026 3:56:15 PM
requirements.txt -a---l 6/3/2026 11:46:45 PM
run.py           -a---l 6/3/2026 11:46:45 PM
```

v. FastAPI Docs Page



VI. Main Streamlit Dashboard Page

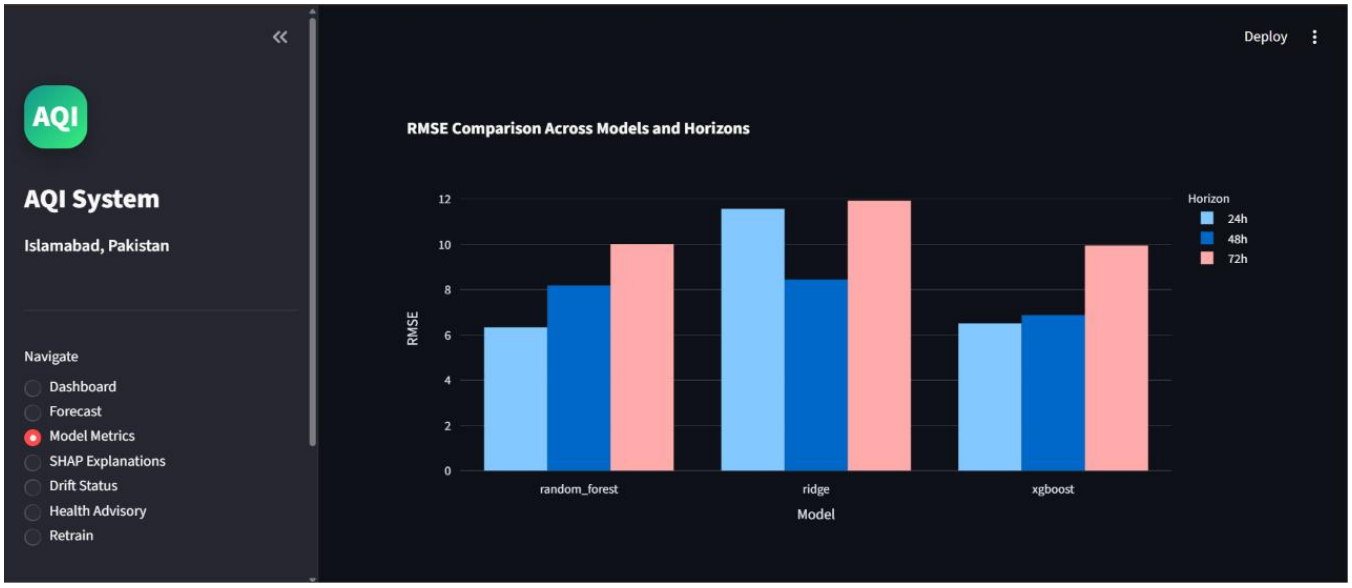

Deploy
AQI
AQI System
Islamabad, Pakistan
Navigate
Dashboard
Forecast
Model Metrics
SHAP Explanations
Drift Status
Health Advisory
Retrain
Islamabad Air Quality Dashboard
Real-time AQI monitoring powered by AQICN and Open-Meteo
68.5
Current AQI
Temperature
31.3°C
Humidity
36%
Wind Speed
2.7 m/s
Moderate: Acceptable. Sensitive individuals should reduce prolonged outdoor exertion.
AQI History — Islamabad
AQI
80
70
60
50
40
May 15
2026
May 17
May 19
May 21
May 23
May 25
May 27
May 29
May 31
Time
Latest Available Pollutant Readings
Pollutant Value / Sub-Index
50
40
30
20
10
0
29.5
43.5
PM25
PM10
Pollutant

VII. AQI Forecast Page

This figure shows the 24h, 48h, and 72h AQI forecast generated by the trained models. This confirms that the forecasting pipeline is connected with the backend and dashboard.

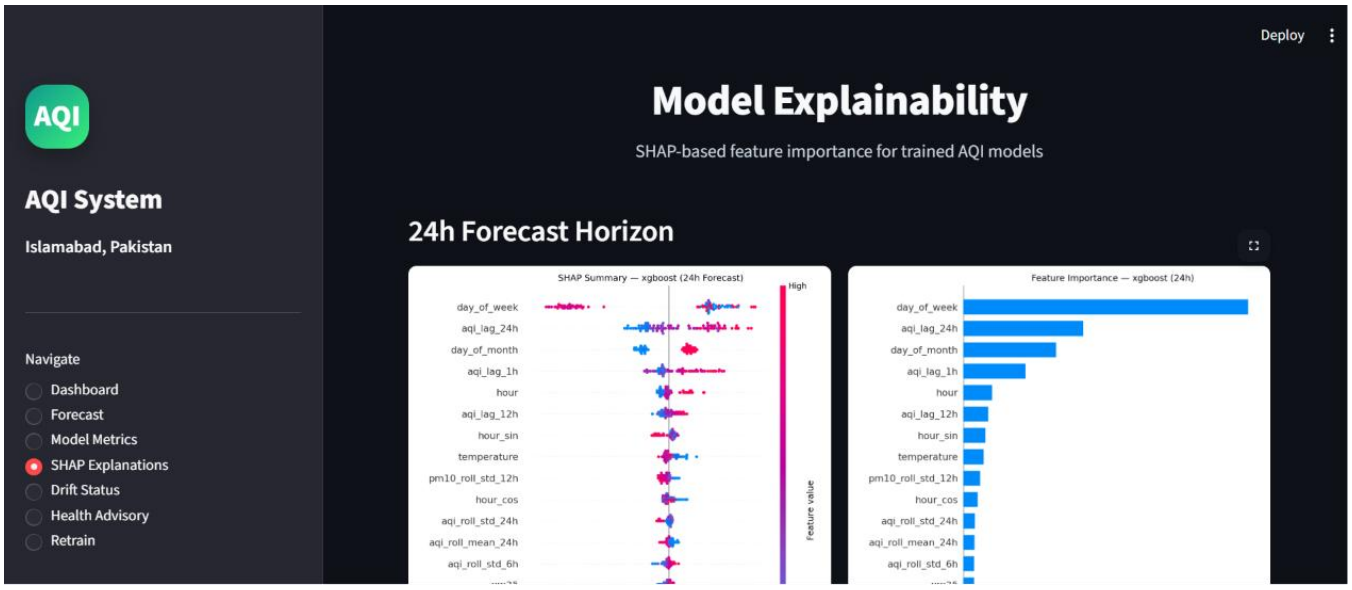
VIII. Model Metrics Page

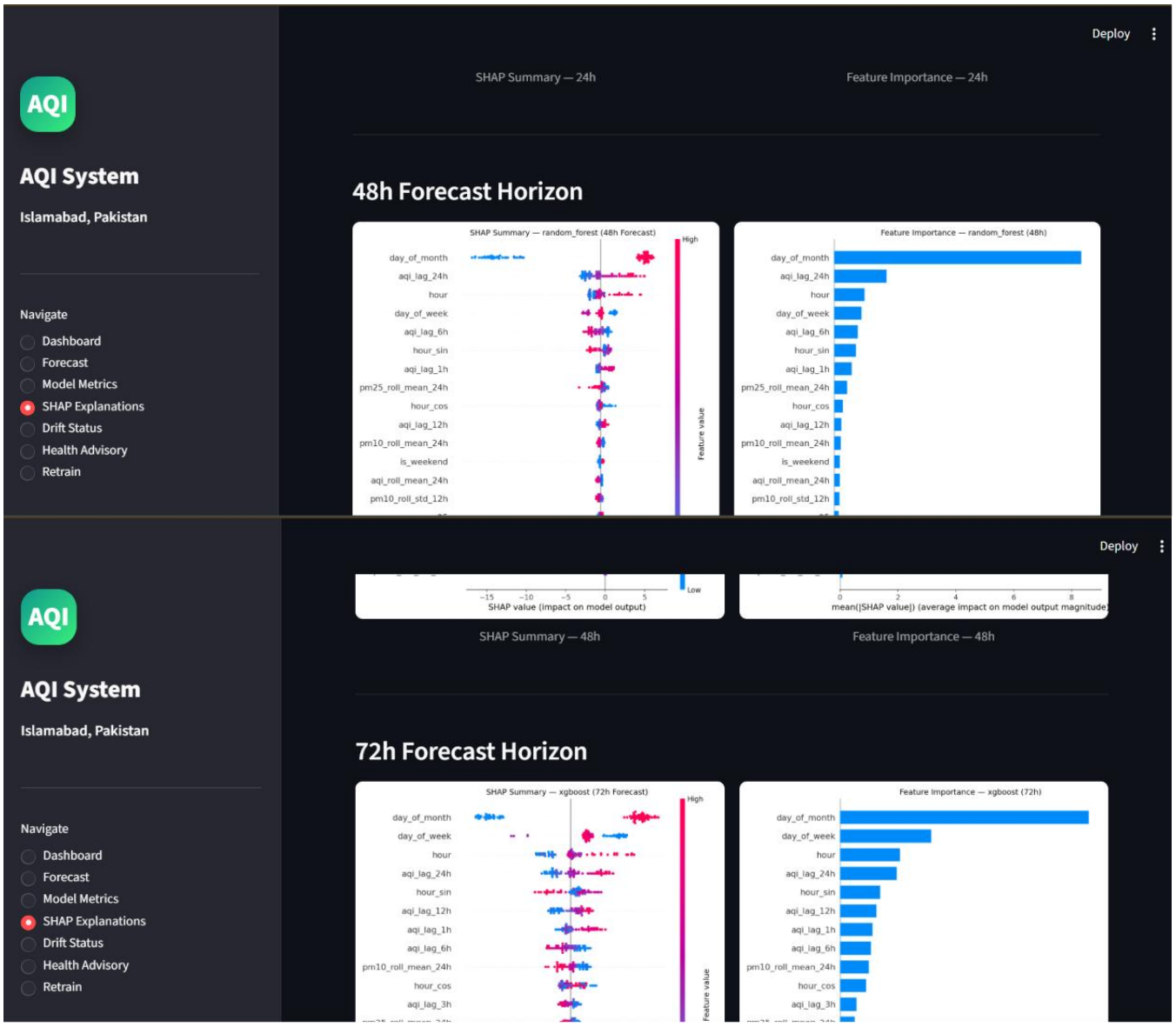
Model	Horizon	RMSE	MAE	R^2	Trained
random_forest	24h	6.3263	4.8249	0.6822	2026-06-06T01:34:37
random_forest	48h	8.1753	6.5973	0.462	2026-06-06T01:34:40
random_forest	72h	10.0043	8.3957	0.2176	2026-06-06T01:34:43
ridge	24h	11.5651	9.0745	-0.0622	2026-06-06T01:34:37
ridge	48h	8.4406	6.9728	0.4265	2026-06-06T01:34:39
ridge	72h	11.9215	9.6371	-0.111	2026-06-06T01:34:42
xgboost	24h	6.5002	4.9043	0.6645	2026-06-06T01:34:39
xgboost	48h	6.8683	5.4897	0.6203	2026-06-06T01:34:42
xgboost	72h	9.9377	8.3384	0.228	2026-06-06T01:34:44



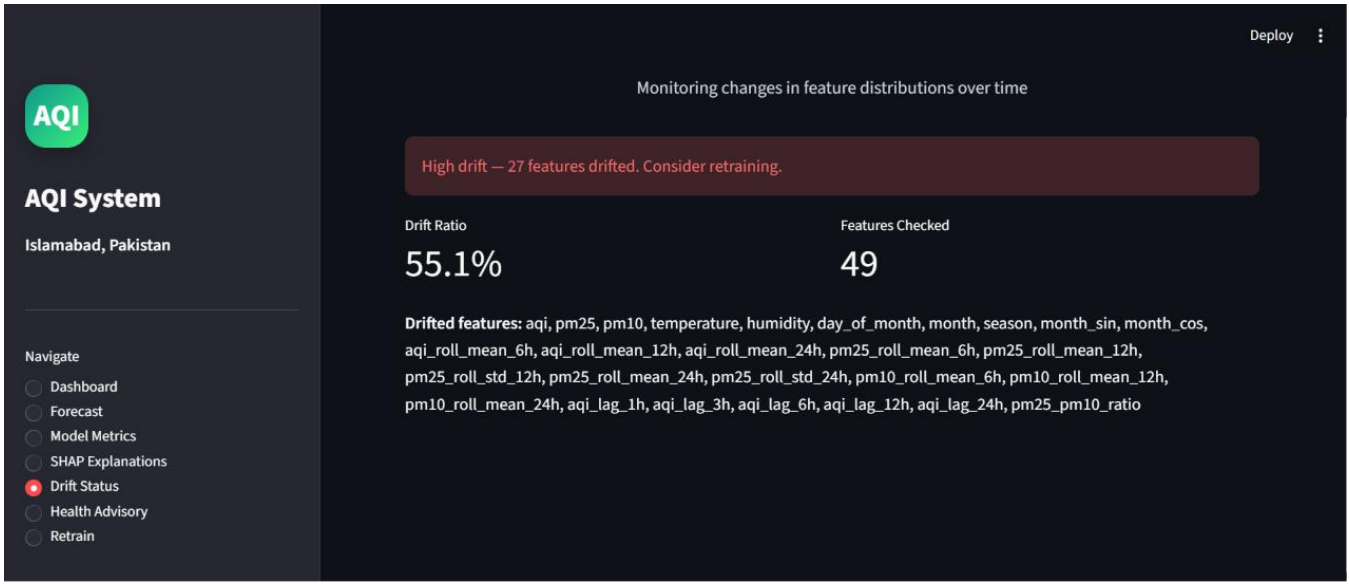
IX. SHAP Explainability Page

Configure for 24h,48h, and 72 hrs:

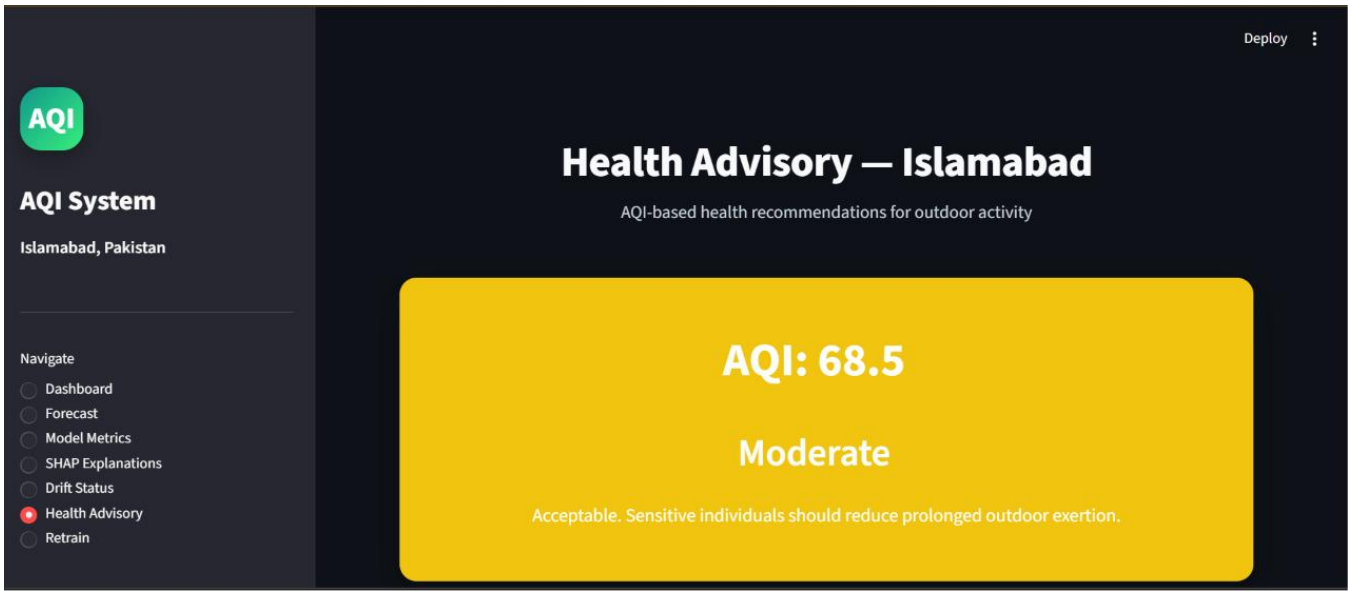


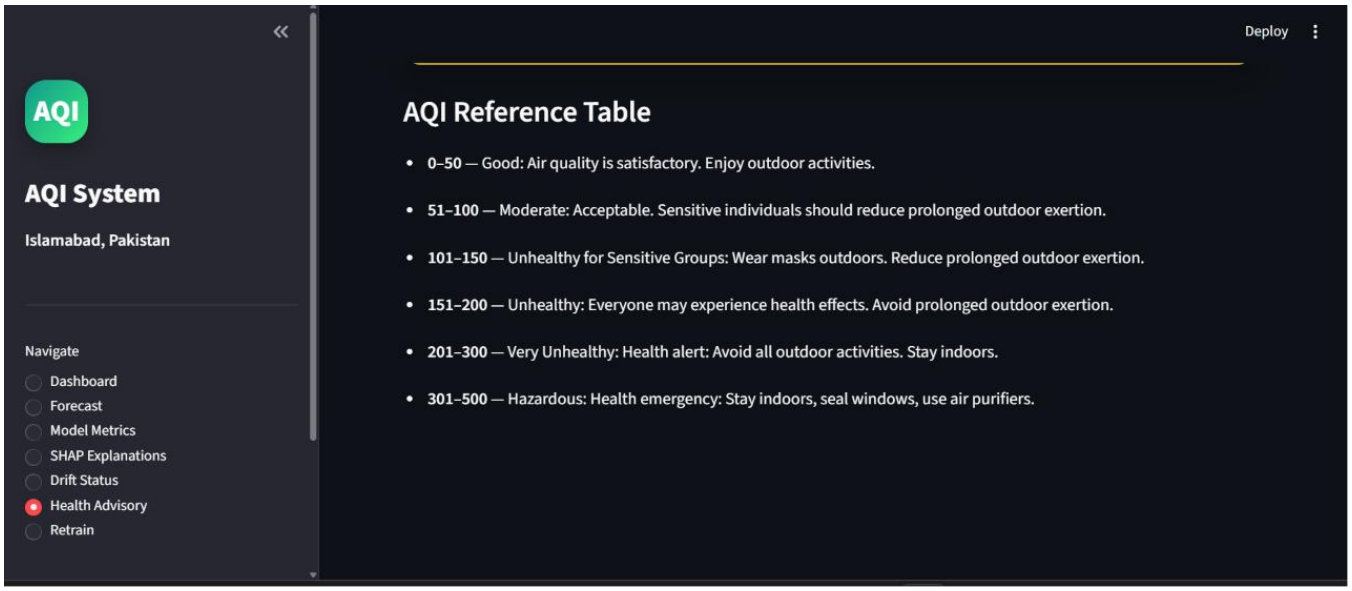

Deploy
AQI
AQI System
Islamabad, Pakistan
Navigate
Dashboard
Forecast
Model Metrics
SHAP Explanations
Drift Status
Health Advisory
Retrain
SHAP Summary — 24h
Feature Importance — 24h
48h Forecast Horizon
SHAP Summary — random_forest (48h Forecast)
Feature Importance — random_forest (48h)
day_of_month
aqi_lag_24h
hour
day_of_week
aqi_lag_6h
hour_sin
aqi_lag_1h
pm25_roll_mean_24h
hour_cos
aqi_lag_12h
pm10_roll_mean_24h
is_weekend
aqi_roll_mean_24h
pm10_roll_std_12h
High
Low
Feature value
SHAP value (impact on model output)
mean(|SHAP value|) (average impact on model output magnitude)
SHAP Summary — 48h
Feature Importance — 48h
72h Forecast Horizon
SHAP Summary — xgboost (72h Forecast)
Feature Importance — xgboost (72h)
aqi_lag_3h

X. Drift Status Confirmation



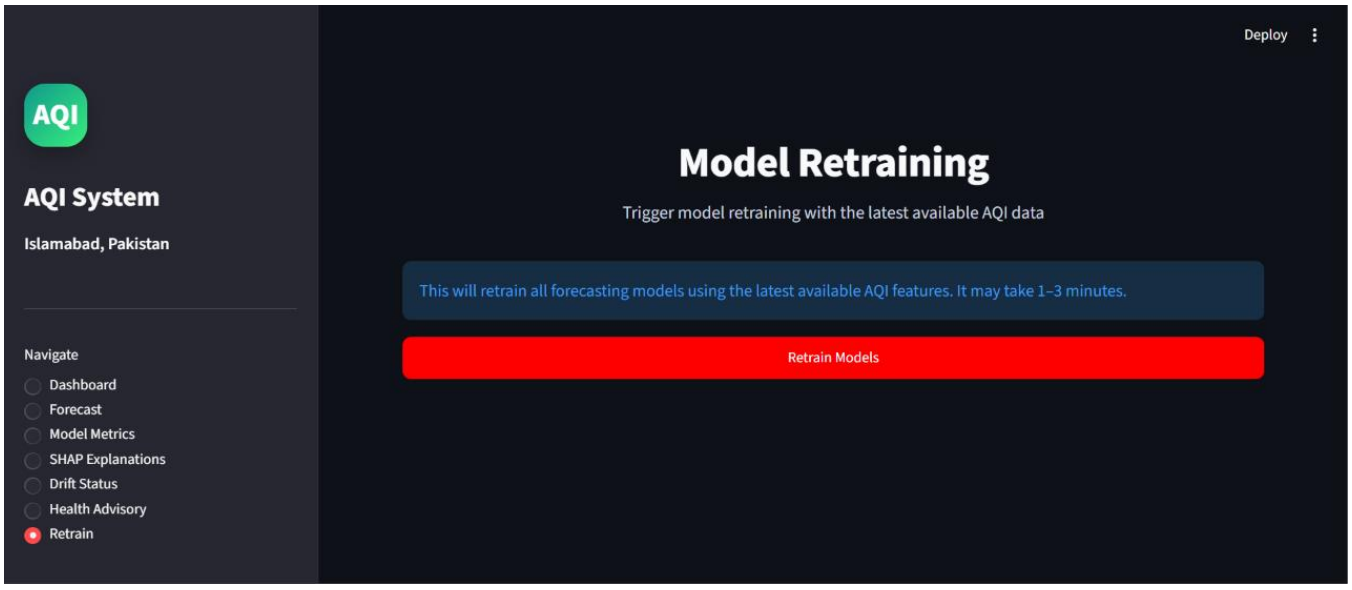
XI. Health Advisory Page





XII. Retrain Page

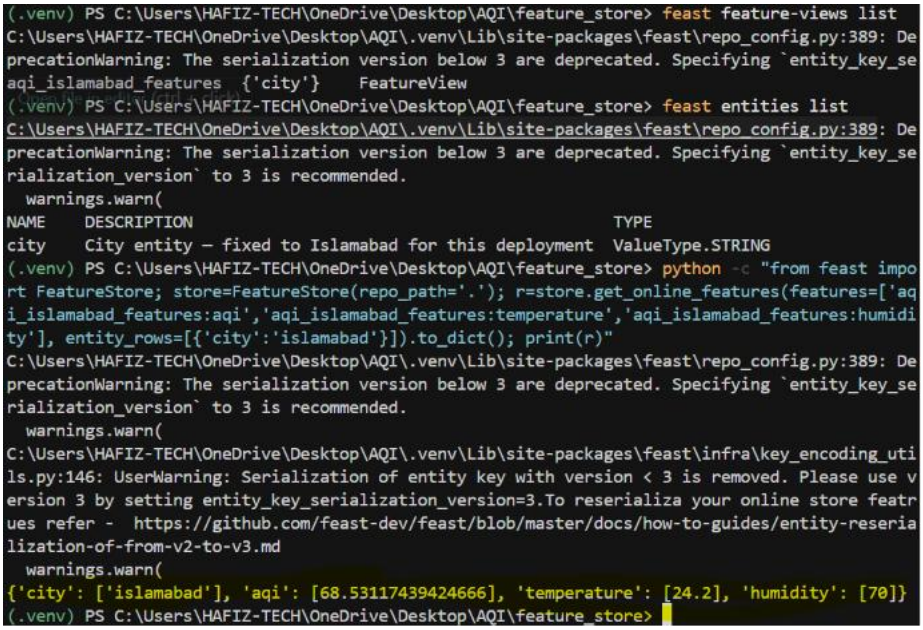
Click the “Retrain model” button to train the modek again in real time.



XIII. Feast Feature Store Verification

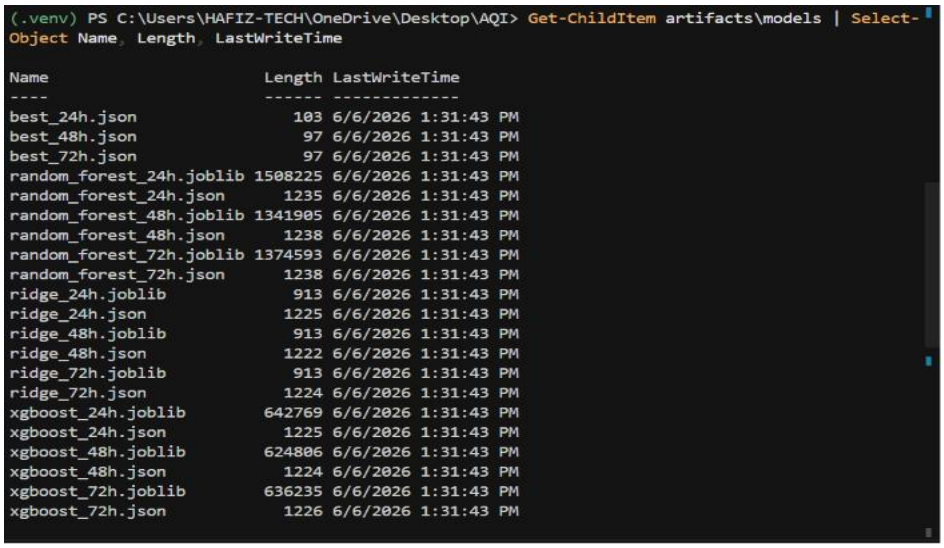
The yellow highlighted portion in the Figure ensures Feast feature store verification showing registered feature view, city entity, and successful online feature retrieval.

The output confirms that the feature view `aqi_islamabad_features` and entity `city` were registered successfully. The online feature retrieval also returned AQI, temperature, and humidity values for Islamabad, which proves that the Feast online store was working.



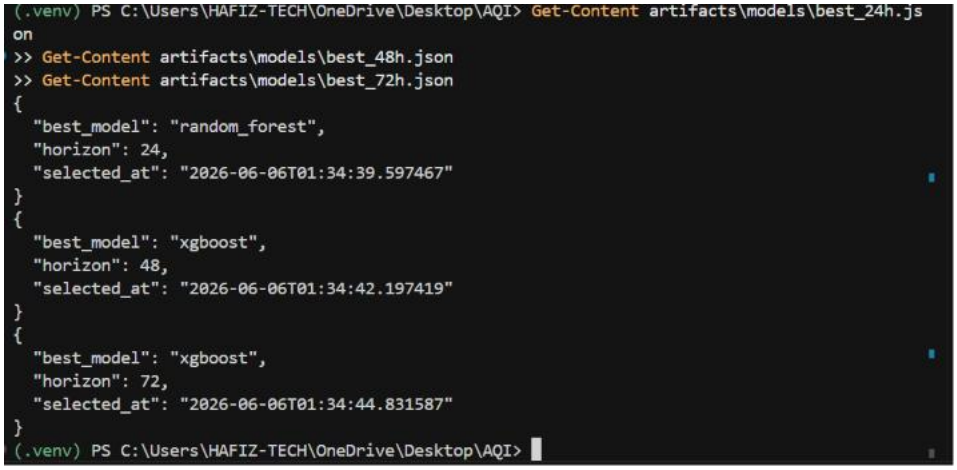
XIV. Model Registry Folder

The model registry folder containing trained model artifacts and metadata files for all three forecasting horizons. The `best_24h.json`, `best_48h.json`, and `best_72h.json` files store the selected best model for each horizon.



XV. Best Model Metadata

Best model selection metadata for each AQI forecasting horizon. The final selected models were Random Forest for the 24-hour forecast and XGBoost for the 48-hour and 72-hour forecasts. These selections were made based on evaluation metrics such as RMSE, MAE, and R^2 score.



18. Conclusion

This internship project was a valuable learning experience for me because it gave me the opportunity to work on a complete machine learning system instead of only building a standalone model. Through this project, I understood how real-world data science projects are structured, how different components are connected, and how important it is to build a reliable end-to-end pipeline.

During this project, I worked with real AQI and weather data, created ingestion and preprocessing pipelines, performed feature engineering, trained multiple machine learning models, integrated a feature store, built a model registry, developed a FastAPI backend, and created a Streamlit dashboard for visualization. I also explored model explainability using SHAP and automated parts of the workflow using GitHub Actions. These steps helped me understand the complete lifecycle of a machine learning project, from raw data collection to model serving and user-facing visualization.

One of the most important learning areas for me was the concept of feature stores. Before this project, I had limited practical understanding of tools such as Feast and Hopsworks. While working on this project, I learned why feature stores are important in machine learning systems, how offline and online feature stores work, and how they help reduce the gap between training and serving. I

implemented Feast in this project because it was lightweight, open-source, reproducible locally, and suitable for an internship-level project. At the same time, studying Hopsworks helped me understand how managed feature store platforms are used in production-level systems.

This project also improved my debugging and problem-solving skills. I faced several issues related to data availability, timestamp handling, feature store materialization, GitHub Actions workflow failures, model artifact paths, dependency installation, dashboard errors, and Git/GitHub version control. Instead of treating these problems as failures, I used them as learning opportunities. I debugged each issue step by step by checking logs, validating outputs, fixing configuration files, testing locally, and then pushing stable changes to GitHub.

Another important learning from this internship was the effective use of AI tools in the development process. I learned how to use AI tools for understanding complex concepts, debugging errors, improving code structure, reviewing project flow, generating documentation ideas, and identifying possible issues in the pipeline. Instead of depending on AI tools blindly, I used them as a support system for learning, validation, and problem-solving. This helped me become more confident in asking the right technical questions, interpreting suggestions, testing solutions, and applying them carefully in my own project.

Another important lesson was that model performance should be judged through actual evaluation metrics rather than assumptions. I trained Ridge Regression, Random Forest, and XGBoost models and selected the best model for each forecast horizon based on RMSE, MAE, and R^2 score. This helped me understand that different models may perform differently depending on the data, forecast horizon, and feature patterns.

Overall, this internship helped me grow from simply understanding machine learning concepts to implementing a more complete machine learning system. It strengthened my practical knowledge of data engineering, feature engineering, MLOps concepts, AI-assisted development, API development, dashboard creation, automation, explainability, and documentation. This project was challenging, but it became one of the most meaningful parts of my learning journey because every issue I solved helped me understand the practical side of building machine learning systems.